Kinect Lite: Cleaning Kinect-Style Depth Images for Better Camera Tracking

A simple median filter vs. the spatial-temporal pipeline of Fu et al. (2012)

Mini Project Report

November 2026

Abstract

Depth cameras such as the Kinect give a distance for every pixel, but the images contain *spikes* (pixels with a badly wrong distance) and *holes* (pixels with no distance). We compare two ways of cleaning the depth image: our own per-frame **9×9 median filter**, and the spatial-temporal denoising pipeline of **Fu et al. (2012)**. Instead of stopping at "how close is the cleaned depth to the truth", we also feed each cleaned sequence into two camera trackers, one that uses depth only and one that uses colour and depth, and measure how far the estimated camera path drifts from the true path. Tests use a synthetic dataset with true depth (ICL-NUIM) and a real-sensor dataset (TUM RGB-D).

Findings. (1) The median halves the depth error on ICL (18.8 vs. 39.2 mm; the paper: 37.7 mm). (2) Which cleaner gives better tracking depends on the tracker: the paper's pipeline gives 4–16× less drift with the depth-only tracker, and the median is best with the colour-plus-depth tracker (0.98 cm vs. 26.0 cm on ICL). (3) Of eight ways to fill the remaining holes, none improves the colour-plus-depth tracker. (4) The median runs in under a minute for 450 frames; the paper's pipeline takes about 26 minutes.

1 Introduction

A camera that moves through a room can work out its own path by lining up each new depth image with the previous one. This is *camera tracking* (also called visual odometry), and it is the first step of 3D scanning and SLAM. Errors in the depth image lead to small errors in each step, and these add up to *drift*.

Kinect-style sensors produce two kinds of depth error:

- **Spikes:** single pixels or small clusters whose depth is wrong by centimetres to metres, mostly at object edges and on far or shiny surfaces.
- **Holes:** pixels where the sensor returns no depth at all, e.g. in the shadow behind a foreground object, or where the surface is too close, too far or too dark.

Fu et al. [1] proposed a denoising pipeline for this kind of depth. It fills holes by searching for matching patches in neighbouring frames, and then smooths with a spatial-temporal bilateral filter. They judged it by visual quality. We ask a more practical question:

Research question

Does cleaning the depth image make camera tracking more accurate, and is the paper's pipeline worth its cost compared with a simple median filter?

Contributions.

1. A re-implementation of Fu et al. that covers all three possible readings of an ambiguous equation in the paper.

2. A comparison against a per-frame 9×9 median, on depth accuracy *and* on tracking accuracy with two different trackers, on synthetic and real data.
3. A comparison of eight hole-filling methods, scored on fill accuracy and on tracking.

2 Methods

2.1 The two depth cleaners

Both cleaners first apply the same **gate**: any depth closer than 0.4 m (below the sensor's working range) is marked as a hole. The gated depth with no further cleaning is our baseline, called *no cleaning (input)*.

Ours: 9×9 median (`src/clean.py`). Each pixel is replaced by the median of the *measured* pixels in its 9×9 window. Holes are ignored when the median is computed and remain holes. Each frame is processed on its own. We chose the median in an earlier filter study (`experiments/filter_study/`) because the noise is mostly spikes: a median removes a spike, while an average spreads it over its neighbours.

Paper: Fu et al. (2012) (`src/paper_pipeline.py`). For each frame t, using frames $t-2 \dots t+2$:

1. **Padding:** small holes are roughly filled with the mean of their measured neighbours; large holes ($\geq$100 px) are treated as out-of-range and left empty.
2. **Block matching:** for each pixel, find the 9×9 block in each neighbouring frame (search radius 6 px) with the lowest mean squared error (MSE).
3. **Hole prediction** (Eq. 7–9 of the paper): each hole gets a weighted average of the matched values, with weight $w_i = \exp(-\mathrm{MSE}_i^2 / 2\sigma_{\mathrm{MSE}}^2)$.
4. **ST-DNBL:** a bilateral filter over frames $t \pm 1$ (motion-compensated), whose depth-similarity term is scaled by the sensor noise model $\Theta(z) = (0.002\, z^2)^2$, so far surfaces are smoothed more.

Settings: $k = 9$, $\sigma_r = 16$, $\sigma_{\mathrm{MSE}} = 10^{-4}\,\mathrm{m}^2$, tuned on ICL frames 600–1400 (no overlap with the test frames).

An ambiguity in the paper. Eq. 7 sums over frames $i \in [-m, m]$, which includes the current frame ($i = 0$). It is unclear how that term should be used, so we implemented three readings: `paper` (as written), `exclude` (skip $i = 0$) and `nonlocal` (search the current frame, but not at zero shift). All three give results within 0.06 cm of each other (Appendix A), because holes are only about 1 % of ICL pixels. The main text reports the `paper` reading.

2.2 Where colour comes in, and where it does not

Neither cleaner uses the colour image

Fu et al. is a depth-only method, and so is our median. Both read only the depth image. The colour (RGB) image is used only *after* cleaning, by one of the two trackers. It is the **same, unmodified image for every method**, so the only thing that differs between the methods is the depth they produce. Any difference in tracking accuracy is therefore caused by the depth cleaning.

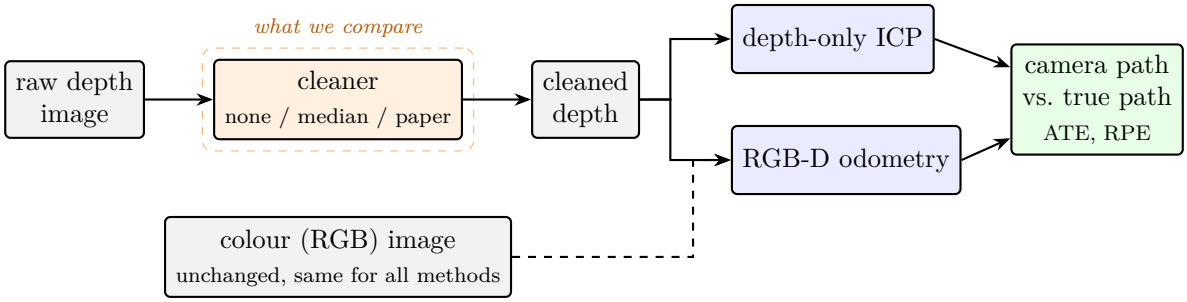


Figure 1: Experiment pipeline. Only the cleaner changes between runs. The colour image skips the cleaner and goes straight to the RGB-D tracker, identical for every method.

Why test with an RGB-D tracker at all? Real systems use both kinds of tracker: depth-only systems (KinectFusion-style ICP) and colour-plus-depth systems (most modern RGB-D SLAM). A depth cleaner is only useful if it helps the tracker it is plugged into. Testing both turned out to matter, since the best cleaner is different for each (Section 3.2).

- **Depth-only ICP**: Open3D point-to-plane ICP between consecutive point clouds, three levels (voxel 4/2/1 cm, max. match distance 10/5/2 cm, 30 iterations each), starting from the previous motion. It uses the depth only.
- **RGB-D odometry**: Open3D hybrid odometry, which combines a photometric (colour) term and a geometric (depth) term. It uses the depth to project *every* pixel into the next frame, so wrong depth values affect it directly.

Both trackers align consecutive frames and chain the motions into a path.

2.3 Data

dataset	type	frames	ground truth
ICL-NUIM living room kt0 [2]	synthetic, simulated Kinect noise	1–300	true depth *and* true camera path
TUM RGB-D fr3 long_office_household [3]	real Asus Xtion sensor	0–299	true camera path (motion capture), no true depth

Table 1: Datasets. Every 2nd frame is tracked (150 frames per sequence). TUM has no true depth, so depth accuracy is measured on ICL only. The camera moves 6.5 mm per tracked step on ICL (0.97 m in total) and 20.7 mm on TUM (3.09 m).

2.4 Metrics

3 Results

3.1 Depth accuracy (ICL)

The median halves the depth error; the paper's pipeline barely changes it. The input's RMSE (187 mm) is almost five times its MAE (39 mm), so the ICL noise is dominated by spikes. The median removes them (RMSE drops to 55 mm), and it lowers the error on every one of the 300 frames. The paper's bilateral filter weights each neighbour by how similar its depth is to the centre pixel, so a spike is only averaged with itself and survives (RMSE 186 mm). The paper's output is smoother, but its error grows with distance (its smoothing widens as $\Theta(z)$ grows), so fewer pixels end up within 1 cm (33.7 % vs. 44.7 %). Figure 2 shows one frame.

metric	meaning	better
MAE (mm)	mean $\lvert d - d^* \rvert$ over pixels with a value, d cleaned and d^* true depth	lower
RMSE (mm)	$\sqrt{\text{mean}\,(d - d^*)^2}$; large errors count more. RMSE $\gg$ MAE means spikes remain	lower
within 1 cm (%)	share of pixels less than 10 mm from the true depth	higher
ATE (cm)	absolute trajectory error: RMS distance between estimated and true camera positions after the best rigid alignment of the whole path. Measures overall drift	lower
RPE (mm)	relative pose error: RMS error of each single step between tracked frames. Measures local accuracy and does not accumulate	lower

Table 2: Metrics. Depth metrics are averaged over the 300 ICL frames; pixels a method leaves as holes are not scored. Differences below about 0.05 cm ATE are within run-to-run noise.

method	MAE (mm)	RMSE (mm)	median error (mm)	within 1 cm (%)
no cleaning (input)	39.19	187.45	13.33	42.19
median 9×9 (ours)	**18.77**	**55.41**	**12.32**	**44.66**
median 9×9 + Telea fill	19.34	57.67	12.36	44.47
paper (Fu et al.)	37.69	186.00	13.42	33.69

Table 3: ICL depth error against the true depth, mean over 300 frames. Bold: best per column.

3.2 Camera tracking

	ICL-NUIM kt0				TUM fr3 long_office_household			
	depth-only ICP		RGB-D odometry		depth-only ICP		RGB-D odometry	
method	ATE	RPE	ATE	RPE	ATE	RPE	ATE	RPE
true depth (best possible)	*0.19*	*0.07*	*0.81*	*2.18*	–	–	–	–
no cleaning (input)	17.07	14.43	23.59	24.20	13.97	11.69	2.14	**5.61**
median 9×9 (ours)	14.94	11.78	**0.98**	**1.59**	13.32	10.86	**2.08**	5.69
median 9×9 + Telea fill	15.18	12.01	1.44	1.91	20.20	12.46	3.13	5.69
paper (Fu et al.)	**0.95**	**2.02**	26.07	21.76	**3.33**	**4.37**	2.10	5.80

Table 4: Camera path error over 150 tracked frames: ATE in cm, RPE in mm. Bold: best per column, not counting the true-depth row, which tracks the noise-free ICL depth and shows the best each tracker can do.

The result flips with the tracker

Depth-only ICP: the paper's pipeline wins clearly (ATE 0.95 vs. 14.94 cm on ICL, 3.33 vs. 13.32 cm on TUM).
RGB-D odometry: the median wins on ICL (0.98 cm, close to the 0.81 cm limit) and ties on TUM; the paper's pipeline does worse than no cleaning on ICL (26.1 cm).

Why the paper wins with depth-only ICP. Point-to-plane ICP discards point pairs that are far apart, so the spikes the paper leaves behind cost little. ICP also relies on surface normals, which need smooth surfaces. The paper's bilateral smoothing gives clean surfaces and good normals. The median leaves the sensor's striped error pattern (Figure 2), which biases every alignment step slightly in the same direction. No single step fails; the small biases add up to drift. With ICP, the median improves only a little on the raw input (14.94 vs. 17.07 cm on ICL).

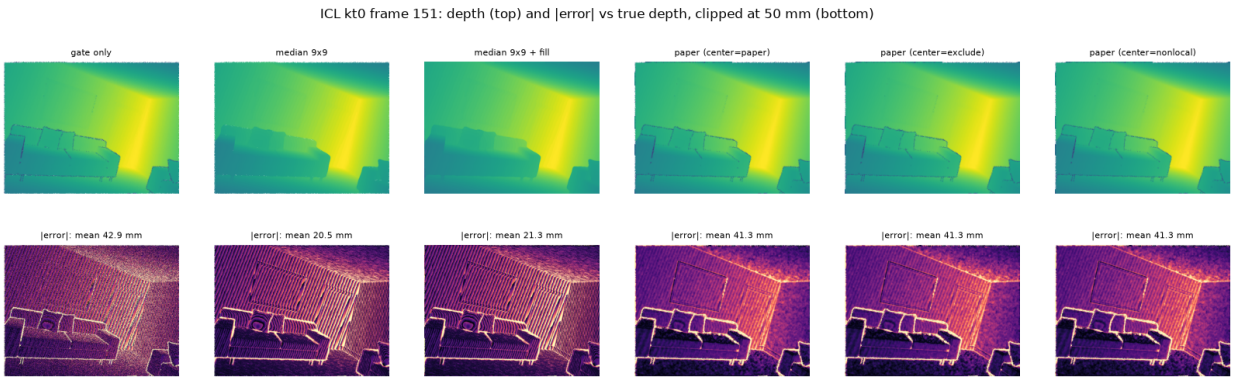


Figure 2: ICL frame 151. Top: depth after each method. Bottom: absolute error against the true depth (brighter = worse, clipped at 50 mm). The median removes the speckled spikes but keeps the sensor's striped error pattern; the paper's output is smooth but keeps the spikes and the edge errors.

Why the median wins with RGB-D odometry. RGB-D odometry uses each pixel's depth to project it into the next frame and then compares colours. A spike sends its pixel to the wrong place, and since nothing discards it, it pulls the estimate. The median removes the spikes, so the tracker gets close to its noise-free result (0.98 vs. 0.81 cm). The paper's output keeps them, and the tracker does slightly worse than with no cleaning at all. Figure 3 shows the effect.

TUM (real sensor). With RGB-D odometry all three main methods land within 0.06 cm of each other (2.08–2.14 cm), so cleaning makes little difference there. This is consistent with the real sensor's noise being less spike-heavy than ICL's simulated noise, but we did not test that directly. The depth-only ICP result (paper 4× better) holds on real data too.

3.3 Should the remaining holes be filled?

After the median, about 1 % of ICL pixels and 16 % of TUM pixels are still holes. We tested eight ways to fill them (`experiments/hole_filling/`). Fill error is measured against the true depth at ICL's real holes.

	fill error	ICL ATE (cm)		TUM ATE (cm)		time per
fill method (after the median)	ICL (mm)	ICP	RGB-D	ICP	RGB-D	frame (TUM)
none (keep holes)	–	**14.94**	**0.98**	13.33	**2.08**	0 ms
Telea inpainting (OpenCV)	87.0	15.18	1.44	20.21	3.13	210 ms
Navier–Stokes inpainting (OpenCV)	22.5	15.36	1.46	**12.03**	3.89	224 ms
nearest measured pixel	22.3	15.05	1.43	17.54	4.16	50 ms
region growing, small holes only	15.5	**14.94**	1.01	13.32	2.23	52 ms
region growing, median of neighbours	22.5	15.15	1.43	15.14	4.08	386 ms
background (far side of the hole)	83.1	15.24	1.80	13.51	3.14	40 ms
temporal (paper's prediction, $t \pm 2$)	**12.2**	15.11	1.24	13.37	2.51	7.9 s

Table 5: Hole-filling methods. The "small holes only" fill fills just 4 % of ICL hole pixels and the temporal fill 88.5 %, which flatters their fill error. Full numbers, including a masked test on both datasets, are in `experiments/hole_filling/results/fills.txt`.

No fill improves RGB-D odometry, on either dataset. A filled pixel is a guess, and the trackers do better ignoring a missing pixel than using a wrong one. The fills that change the least (small holes only, temporal) do the least harm. Telea inpainting, the obvious first choice, is among the least accurate (87 mm), because it blends the near and far sides of a depth edge. The one improvement, Navier–Stokes with ICP on TUM (12.03 vs. 13.33 cm), is reversed on ICL (15.36 vs. 14.94 cm), so we do not count it.

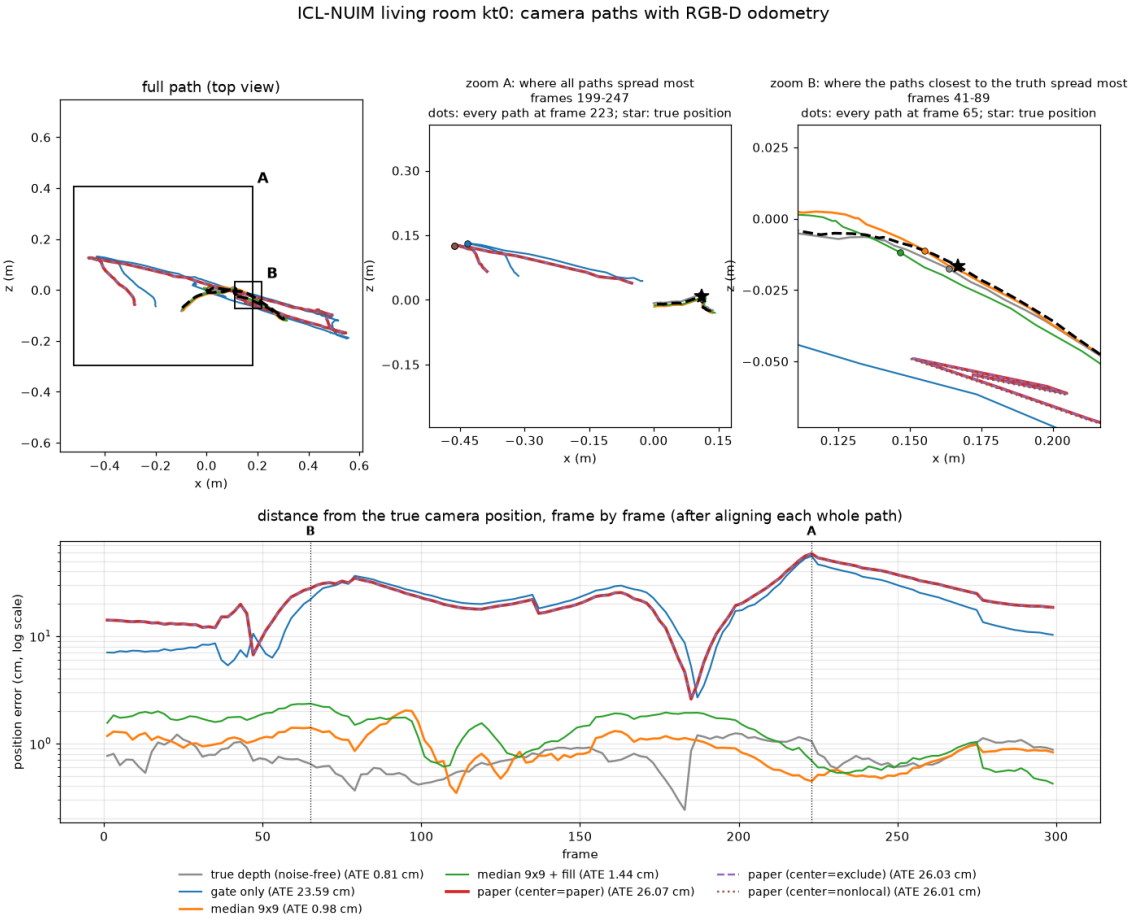


Figure 3: ICL, RGB-D odometry. Top: estimated camera paths seen from above, with two zoomed regions. Bottom: position error per frame (log scale). The median (orange) stays near the true path; the paper's output (red) drifts as much as the uncleaned input (blue). The three paper readings coincide.

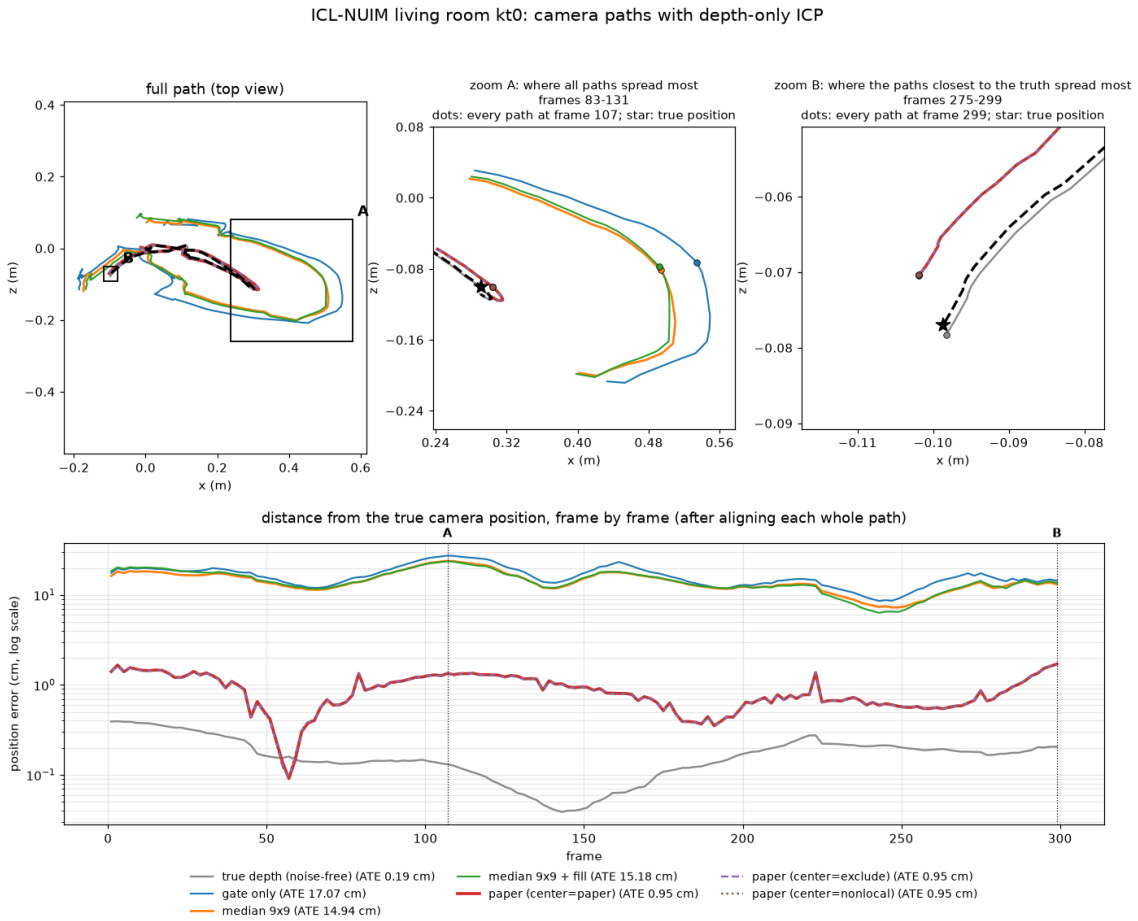


Figure 4: ICL, depth-only ICP, panels as in Figure 3. Here the order is reversed: the paper's output (red) tracks closely, while the median and the input drift.

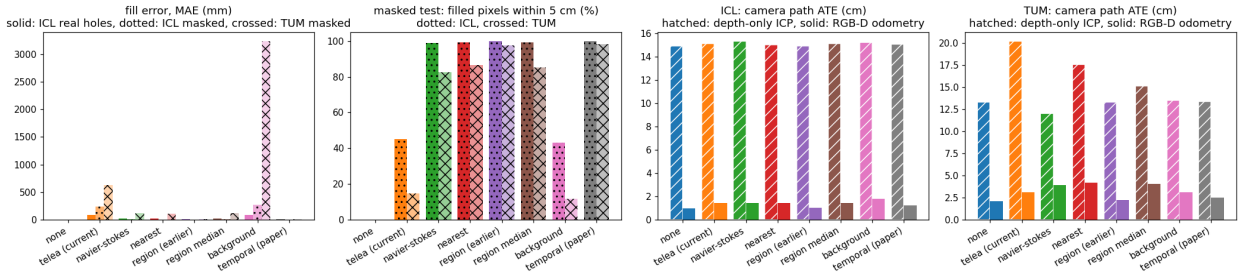


Figure 5: Fill error (left two panels) and camera path ATE (right two panels) for each fill method.

4 Discussion

Depth accuracy does not predict tracking accuracy. The median is far more accurate per pixel, yet it loses badly with depth-only ICP. What ICP needs is smooth surfaces with consistent normals; what RGB-D odometry needs is the absence of wild outliers. Judging a depth cleaner by MAE alone, or by how the output looks, would give the wrong recommendation for one of the two trackers.

Cost. For 450 frames the median takes under a minute; the paper's pipeline takes about 26 minutes, mostly in block matching. The median can run in real time; the paper's pipeline as implemented here cannot.

Limitations.

- One sequence per dataset, 150 tracked frames each. The large differences (4–16× in ATE) are robust; differences of a few millimetres are not.
- ICL's noise is simulated. The real sensor (TUM) shows much smaller differences between cleaners with RGB-D odometry.
- TUM has no true depth, so depth accuracy is measured on synthetic data only.
- The trackers are frame-to-frame with no loop closure or keyframes; a full SLAM system would correct part of the drift.
- Our re-implementation of Fu et al. may differ from the authors' code in details the paper does not give; we covered the main ambiguity (Eq. 7) with three readings.

5 Conclusion

if your system uses …	use	why
RGB-D odometry (colour + depth)	**9×9 median, no hole filling**	error halved, best or equal-best tracking, <1 min
depth-only ICP	**Fu et al. pipeline**	4–16× less drift, but about 26 min
filled depth for display	median + Navier–Stokes or nearest	about 4× more accurate than Telea

A simple per-frame median filter is the better default for colour-plus-depth tracking: it is more accurate, much faster, and does not need future frames. The paper's spatial-temporal pipeline is worth its cost only when the tracker uses depth alone. Filling holes does not help tracking; if filled depth is needed, avoid Telea inpainting.

Reproducing the results

```
pip install -r requirements.txt                              # Python 3.12
python src/icl_compare.py --start 1 --frames 300 --step 2  # ICL
python src/tum_compare.py --start 0 --frames 300 --step 2  # TUM (Kaggle)
python experiments/hole_filling/compare_fills.py --frames 300 --step 2
cd report; pdflatex report.tex; pdflatex report.tex        # this report
```

Raw numbers are in:

- `src/results/icl_compare.txt` and `src/results/tum_compare.txt`
- `experiments/hole_filling/results/fills.txt`

References

[1] J. Fu, S. Wang, Y. Lu, S. Li, W. Zeng, "Kinect-like depth denoising," in *Proc. IEEE ISCAS*, 2012.

[2] A. Handa, T. Whelan, J. McDonald, A. J. Davison, "A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM," in *Proc. IEEE ICRA*, 2014.

[3] J. Sturm, N. Engelhard, F. Endres, W. Burgard, D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ IROS*, 2012.

A The three readings of Eq. 7

	ICL MAE	ICL ICP		ICL RGB-D		TUM ICP		TUM RGB-D	
reading	(mm)	ATE	RPE	ATE	RPE	ATE	RPE	ATE	RPE
`paper`	37.69	0.95	2.02	26.07	21.76	3.33	4.37	2.10	5.80
`exclude`	37.69	0.95	2.02	26.03	21.76	3.31	4.37	2.10	5.79
`nonlocal`	37.69	0.95	2.02	26.01	21.75	3.31	4.37	2.12	5.80

Table 6: The three readings of the hole-prediction equation give the same results to within 0.06 cm ATE.

B TUM camera paths

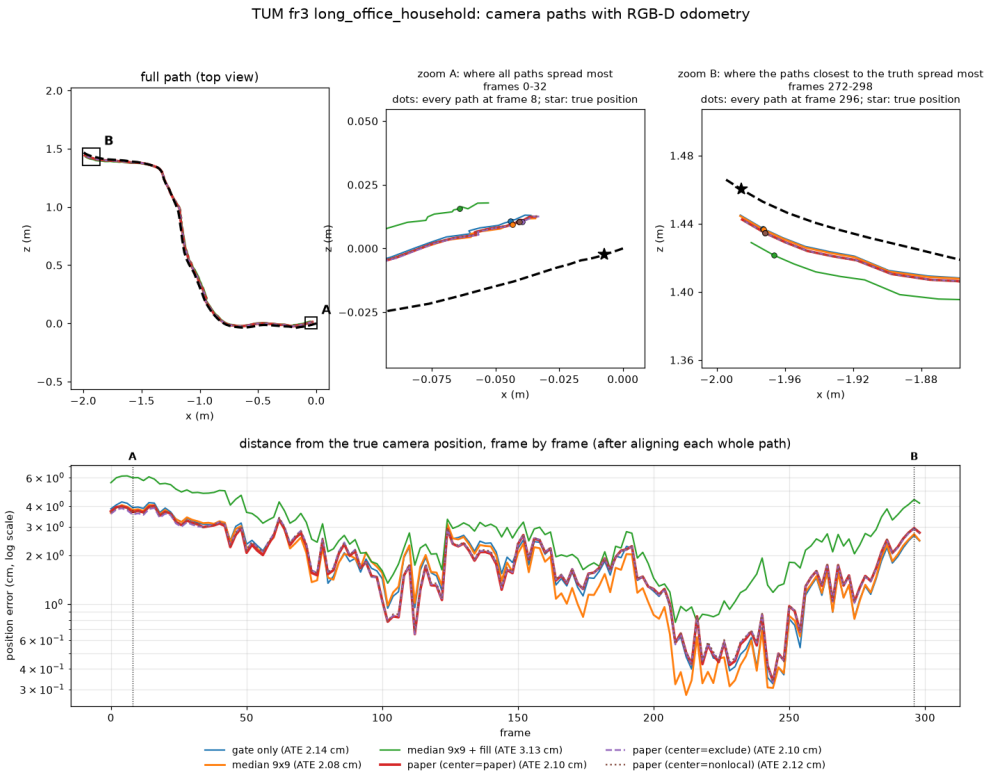


Figure 6: TUM, RGB-D odometry: input, median and paper stay close together (ATE 2.08–2.14 cm); the median with Telea fill (green) drifts more (3.13 cm).

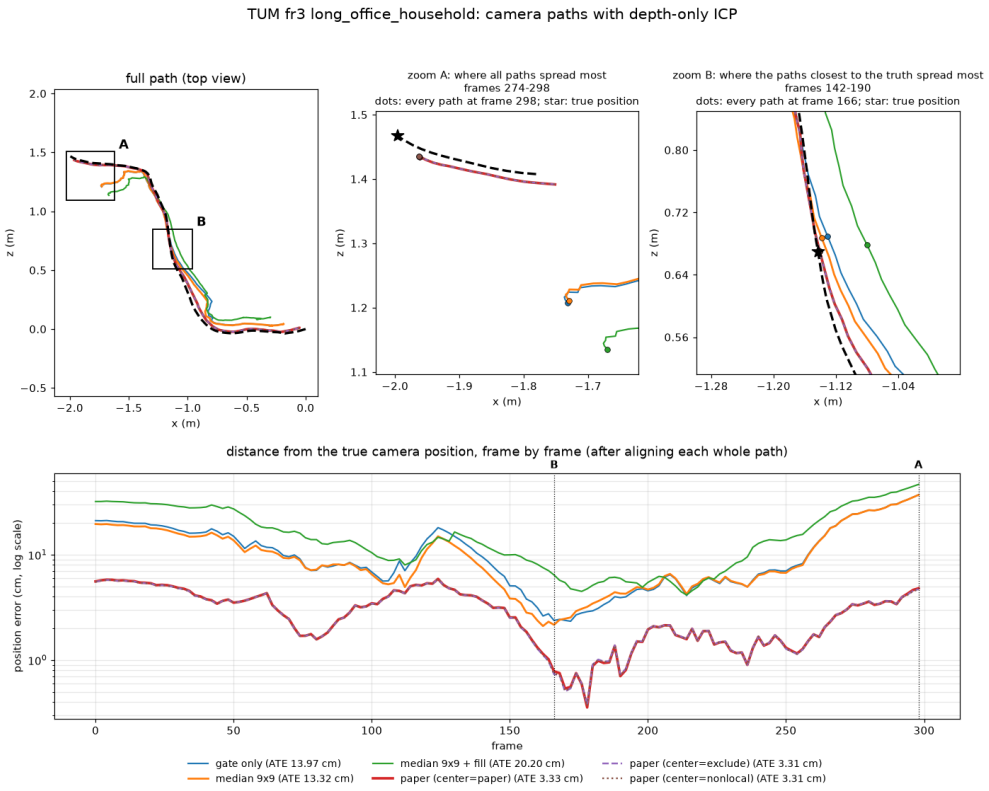


Figure 7: TUM, depth-only ICP: the paper's output (red) drifts much less than the median and the input.